

Write a lex program to recognize telephone operators in Bangladesh.

```
%{
#include<stdio.h>
%}

%%

"015"[0-9]{8}    {printf("Teletalk: %s\n",yytext);}
"016"[0-9]{8}    {printf("Airtel: %s\n",yytext);}
"017"[0-9]{8}    {printf("Grameenphone:
%s\n",yytext);}
"018"[0-9]{8}    {printf("Robi: %s\n",yytext);}
"019"[0-9]{8}    {printf("Banglalink: %s\n",yytext);}
.|\\n;
%%

int yywrap()
{
return 1;
}

int main()
{
yylex();
return 0;
}
```

Write a lex program to count the characters, words and lines in a given C program.

```
%{
#include <stdio.h>
int char_count = 0, word_count = 0, line_count = 0;
}%

%%
[a-zA-Z0-9_]+      { word_count++; char_count +=
yyleng; }
.|\\n              { char_count += yyleng; if (*yytext
== '\\n') line_count++; }
%%

int yywrap()
{
return 1;
}

int main() {
    yylex();
    printf("Characters: %d\\nWords: %d\\nLines: %d\\n",
char_count, word_count, line_count);
    return 0;
}
```

Write a lex program to identify integer, floating point, exponential and complex numbers.

```
%{
#include<stdio.h>
%}

%%

[+-]?[0-9]+ {printf("Integer: %s\n",yytext);}
[+-]?[0-9]+\.[0-9]+([eE][+-]?[0-9]+)?
{printf("Floating-Point: %s\n",yytext);}
[+-]?[0-9]+([eE][+-]?[0-9]+)
{printf("Exponential: %s\n",yytext);}
[+-]?[0-9]+\.[0-9]+([eE][+-]?[0-9]+)?[-+]?[0-9]+\.[0-9]+[i]
{printf("Complex: %s\n",yytext);}
.\n      ;
%%

int yywrap()
{
    return 1;
}

int main()
{
    yylex();
    return 0;
}
```

Write a lex program to identify and count the spaces and comments in a given C program.

```
%{
#include<stdio.h>
int comment= 0;
int space= 0;
}%

%%
[ ]+      {space+=yyleng;}
"//".*    {comment++;}
"/*"([^\]|\\*+[^*/])*"*/"      {comment++;}
.|\\n     ;
%%

int yywrap()
{
    return 1;
}

int main()
{
    yylex();
    printf("No of Comment: %d\\n",comment);
    printf("No of Space: %d\\n",space);
    return 0;
}
```

Write a lex program to recognize and count all of the identifiers in a C program

```
%{
#include <stdio.h>
#include <string.h>
int count = 0;
%}

IDENTIFIER [a-zA-Z_][a-zA-Z0-9_]*
KEYWORD
int|float|char|double|long|short|void|return|if|else|while|
for|do|switch|case|break|continue|struct|union|typedef
%%

{KEYWORD} ; // Ignore keywords
{IDENTIFIER} {
    count++;
    printf("Identifier: %s\n", yytext);
}
.|\\n ; // Ignore everything else
%%

int yywrap()
{
    return 1;
}

int main()
{
    yylex();
    printf("\\nTotal Identifiers (excluding keywords):
%d\\n", count);
    return 0;
}
```

Write a lex program to recognize “to be” verbs from a given paragraph.

```
%{
#include<stdio.h>
#include<string.h>
int count= 0;
%}

TO_BE am|is|are|was|were|been|have|has|be

%%

.*({TO_BE}).*    {
    count++;
    printf("To-Be Verb Found: %s\n", yytext);
}

.|\\n    ;

%%

int yywrap() {
    return 1;
}

int main() {
    yylex();
    printf("Total No of To-Be Verbs: %d\n", count);
    return 0;
}
```

Write a lex program to identify whether a given sentence is simple/ complex/ compound.

```
%{
#include <stdio.h>
%}

COMPLEX_KEYWORDS that|since|because
COMPOUND_KEYWORDS and|or|but

%%

.*({COMPLEX_KEYWORDS}).* { printf("Complex
Sentence: %s\n", yytext); }
.*({COMPOUND_KEYWORDS}).* { printf("Compound
Sentence: %s\n", yytext); }

.|\\n ; // Ignore all other characters

%%

int yywrap() {
    return 1;
}

int main() {
    yylex();
    return 0;
}
```